

Trabajo Práctico Integrador
GESTIÓN DE DATOS DE PAÍSES

Alumnos

Oliverio Arce, Marcos Magnello

Ambos integrantes pertenecen a la comisión 8.

**TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN - UNIVERSIDAD
TECNOLÓGICA NACIONAL.**

Programación 1

15 de junio de 2026

TABLA DE CONTENIDO

Introducción	3
Marco teórico	3
Desarrollo	6
Conclusión	16

INTRODUCCIÓN

El presente trabajo fue realizado en el marco de la materia Programacion 1, de la Tecnicatura Universitaria en Programación a distancia. El objetivo consistió en el desarrollo de un programa en el lenguaje de programación Python para analizar datos partiendo de un archivo .csv (comma separated values), que permita analizar dichos datos y procesarlos adecuadamente. Dado que se trata de un archivo con países y datos básicos de los mismos, se trabajó para que el programa permita realizar distintas búsquedas, filtros, y demás análisis personalizados. El trabajo permitió a los alumnos practicar y afianzar conceptos aprendidos en el transcurso de la materia, y volcarlos en un proyecto real, simulando condiciones de trabajo de un ambiente laboral, tales como trabajo en equipo, reparto de tareas, establecimiento de plazos límites. En el presente documento se encuentran desarrollados los conceptos teóricos que sustentan el desarrollo del proyecto.

MARCO TEÓRICO

Listas

Una lista, definida en el ámbito de las ciencias de la computación, es una estructura de datos que permite almacenar más de un dato dentro de ella. Dentro del lenguaje Python, las listas permiten almacenar distintos tipos de datos, lo que le da una flexibilidad ideal para trabajar con ellas. Las listas tienen dos características principales que hacen a su funcionalidad y la distinguen de otras estructuras de datos. Son mutables, es decir, que se pueden modificar en el transcurso del programa, y son ordenadas, es decir, que se puede acceder a ellas mediante el índice que les corresponde. Son declaradas con corchetes [] y dentro de ellos se introducen los elementos de la lista, separados por comas. Como se mencionó, los elementos dentro de la lista pueden ser de cualquier tipo, booleanos, enteros, flotantes, cadenas de texto, e incluso otras listas u otras estructuras de datos, lo que permite crear estructuras más complejas como las matrices. Para acceder a los elementos de la lista se utilizan los índices, con la característica de que el primer elemento siempre tendrá de índice el 0 (cero), el segundo el 1 (uno) y así sucesivamente. También se pueden utilizar los índices negativos para recorrer la lista de forma invertida, siendo el -1 (menos uno) el último elemento, y así sucesivamente. Al ser una secuencia ordenada, las listas pueden ser recorridas mediante un bucle for o while.

Diccionarios

Un diccionario es una estructura de datos que organiza la información mediante pares clave.valor, donde cada clave es un identificador único asociado a un valor correspondiente. A diferencia de las listas, que almacenan elementos en posiciones secuenciales, los diccionarios permiten el acceso directo a un valor a través de su clave, sin necesidad de recorrer la estructura completa. En Python, los diccionarios son mutables –admiten la incorporación, modificación y eliminación de elementos luego de su creación– y aceptan valores de cualquier tipo de dato, incluidos números, cadenas, listas u otros diccionarios. A partir de la versión 3.7 del lenguaje, conserva el orden de inserción de sus elementos.

Su declaración se realiza mediante llaves “{}”, especificando cada par con la sintaxis clave: valor y separando los pares con comas. También puede crearse un diccionario vacío usando la función “dict()”. El acceso a un valor se realiza indicando la clave entre corchetes: `diccionario[clave]`.

Funciones

Las funciones son definidas como bloques de código reutilizables que cumplen una tarea específica. Idealmente, las funciones deben realizar una sola tarea, cumpliendo con el principio de responsabilidad única (SRP). Dado que en el entorno de la programación es normal encontrarse con tareas que deban ser realizadas varias veces, es lógico contar con una estructura que permita reutilizar el código para no tener que escribirlo varias veces cumpliendo con el principio *Don't Repeat Yourself (DRY)*. De este modo, con las funciones es posible modularizar el código, haciéndolo más claro y difícil de entender. Las funciones en computación guardan similitudes con las funciones matemáticas, ya que reciben uno o más datos, llamados parámetros, (o ninguno) realizan un procesamiento sobre dichos datos, y devuelven un valor determinado. Son declaradas con la instrucción *def*: y luego en el bloque de código subsiguiente, respetando la indentación, que es la forma que tiene Python de distinguir bloques de código, se introduce todo el procesamiento que realiza la función. Finalmente, la instrucción *return* devuelve el resultado de esa función. También es posible declarar funciones sin dicha instrucción, que se denomina funciones “ciegas”.

Condicionales

Las estructuras condicionales son mecanismos de control que permiten modificar el flujo de ejecución de un programa en función de si una condición lógica se cumple o no. En Python, se implementan mediante las sentencias *if*, *elif* y *else*: la primera evalúa una condición y ejecuta un bloque de código si el resultado es verdadero (*true*); la segunda permite encadenar condiciones alternativas cuando hay mas de dos escenarios posibles; la tercera define un bloque que se ejecuta cuando ninguna de las condiciones anteriores se cumple. Las condiciones se construyen a partir de operadores relacionales (*=*, *!=*, *>*, *<*, *>=*, *<=*) y operadores lógicos (*and*, *or*, *not*), que devuelven siempre un valor booleano. En Python, la delimitación de los bloques condicionales no se realiza mediante llaves sino mediante indentación, siendo la convención oficial el uso de cuatro espacios por nivel.

Ordenamientos

Dado que en programación lo más común es trabajar con determinados datos y procesarlos, una tarea muy importante que deben realizar los programas desarrollados es el ordenamiento. Para esto se echa mano de los denominados algoritmos de ordenamiento, que acomodan datos siguiendo algún principio preestablecido. El ordenamiento, entonces, es una tarea fundamental de los programas de computadora, ya que facilitan la búsqueda, visualización, y filtrado de los datos. Un ejemplo muy utilizado es el algoritmo burbuja, que recorre una estructura de datos y compara los datos adyacentes hasta tenerlos completamente ordenados en base a algún criterio, por ejemplo, números de menor a mayor.

Estadísticas Básicas

La estadística descriptiva es la rama de la matemática que permite resumir, organizar y analizar un conjunto de datos mediante indicadores numéricos. En el contexto de este proyecto, los indicadores relevantes son los siguientes.

El máximo y el mínimo representan respectivamente el valor más alto y el más bajo dentro de un conjunto de datos. El promedio o media aritmética se obtiene dividiendo la suma de todos los valores entre la cantidad de elementos, y expresa el valor central respecto del conjunto. El conteo simplemente indica cuántos elementos pertenecen a una categoría o conjunto determinado.

En Python, estos cálculos se implementan mediante funciones incorporadas de la biblioteca estándar. La función *max()* devuelve el valor máximo de un iterable, *min()* el mínimo, *sum()* calcula la suma total de los elementos y *len()* retorna la cantidad de elementos. Para obtener el promedio se combina *sum()* y *len()* mediante la expresión *sum(lista) / len(lista)*. Cuando se trabaja con listas de diccionarios, como en este proyecto, se utiliza el parámetro *key* en *max()* y *min()* para indicar sobre que campo realizar la comparación.

Archivo CSV

Un archivo CSV (Comma Separated Values o Valores Separados por Coma) es un tipo de archivo de texto plano diseñado para almacenar datos en formato tabular. Su estructura organiza la información en filas y columnas: cada línea del archivo representa una fila, y los valores dentro de ella se separan por comas. La primera línea suele contener los encabezados, es decir, los nombres de cada columna. Esta simplicidad lo convierte en un formato ampliamente utilizado en el desarrollo de software por su interoperabilidad entre distintos sistemas, su ligereza respecto a otros formatos y su capacidad de brindar persistencia a los datos, que de otro modo se perderían al finalizar la ejecución del programa.

En Python, el manejo de archivos CSV se realiza a través del módulo `csv` de la biblioteca estándar. Para la lectura, se utilizan `csv.reader`, que interpreta cada fila como una lista o `csv.DictReader`, que la transforma directamente en un diccionario usando los encabezados como claves. Para la escritura se emplean sus equivalentes `csv.writer` y `csv.DictWriter`. En todos los casos se recomienda abrir el archivo mediante el bloque `with open()`, lo que garantiza el cierre automático del recurso incluso ante errores, y especificar el parámetro `encoding="utf-8"` para asegurar la correcta interpretación de caracteres especiales como tildes y ñes.

DESARROLLO

Para el desarrollo del programa, se optó por un diseño de software basado en la Modularización (ya que los diversos acontecimientos que acontecieron durante la última semana en relación a la consigna del trabajo, derivó en que concretemos el trabajo de esta manera) y el principio de separación de responsabilidades. En lugar de concretar toda la lógica en un único archivo, el sistema se subdividió en cuatro módulos independientes (`main.py`, `archivos.py`, `modificar_registros.py` y [consultas.py](#))

A continuación, se detalla analíticamente el funcionamiento, flujo, control de errores y validación de cada componente del sistema.

Módulo Principal (main.py)

Este módulo actúa como el director de una orquesta. Inicia cargando la estructura de datos en memoria y luego administra el ciclo de vida del programa mediante un bucle interactivo (*while True*)

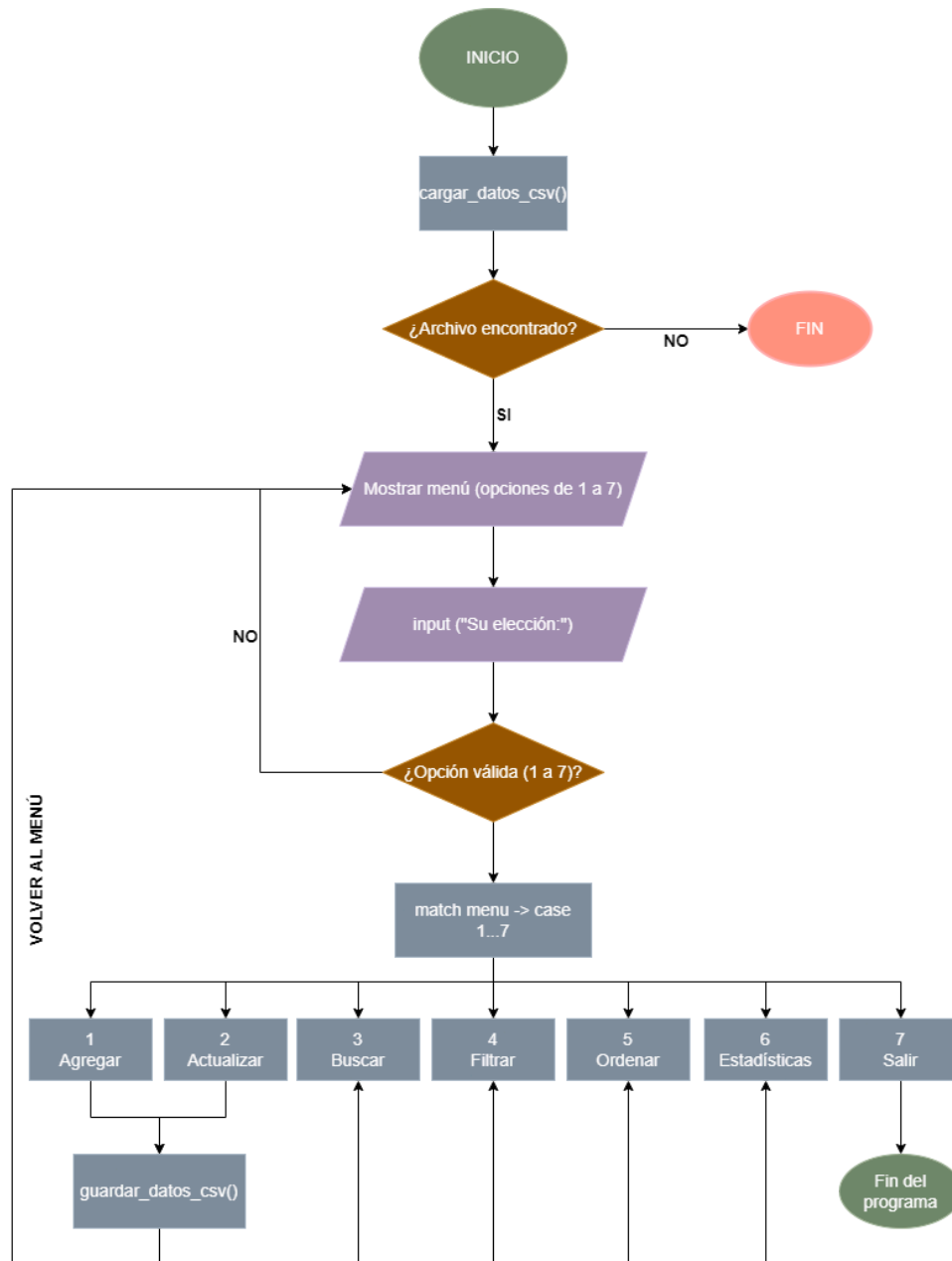


Fig 1: Diagrama de flujo de [main.py](#)

```

Bienvenido al sistema de gestion de datos geograficos. Ingrese la opcion para realizar la opcion que
desees:

===== MENÚ PRINCIPAL =====
1. Agregar un país al registro.
2. Actualizar datos de población y superficie.
3. Buscar país.
4. Búsqueda avanzada por filtro.
5. Lista de países ordenada.
6. Estadísticas.
7. Salir.
=====

Su eleccion: █

```

Fig 2: Menú principal en consola

```

Su eleccion: a
No has ingresado una opcion valida. Ingrese solamente un numero entre 1 y 7.

```

Fig 3: Menú e ingreso invalido (letra)

```

Su eleccion: 8
No has ingresado un numero valido. Ingrese una opcion entre 1 y 7

```

Fig 4: Menú e ingreso fuera de rango

La robustez de la interfaz radica en la implementación de un doble bucle repetitivo anidado. El bucle interno tiene como única responsabilidad asegurar que el usuario provea una opción limpia y valida. Para mitigar los fallos críticos por ingresos alfanuméricos en solicitudes enteras se envolvió la captura del `input()` en una estructura de control de excepciones `try-except` orientada específicamente a capturar la excepción `ValueError`. Una vez garantizado el tipo de dato y verificado que se encuentra dentro del rango de opciones `range(1, 8)`, se rompe el ciclo secundario mediante un `break` y se delega el flujo a una estructura selectiva múltiple `match-case`.

Módulo archivos.py

Este módulo opera como la capa de persistencia de datos (Entrada/Salida), comunicando la memoria volátil de Python con el medio de almacenamiento físico en formato plano CSV.

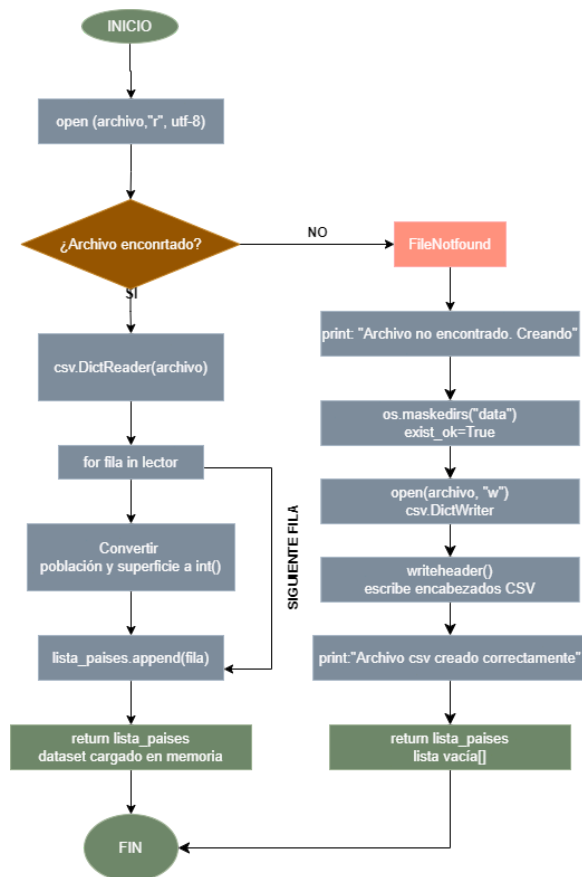


Fig 5: Diagrama de flujo de la función cargar_datos_csv

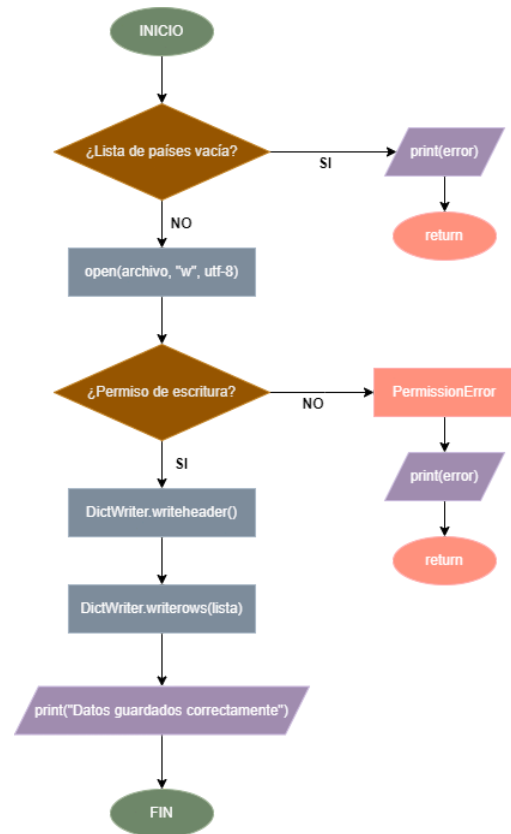


Fig 6: Diagrama de flujo de la función guardar_datos_csv

```

datos_prueba.csv > data
nombre,poblacion,superficie,continente
Argentina,46000000,2780400,América
Japón,125800000,377975,Asia
Brasil,213993437,8515767,América
  
```

Fig 7: Formato del archivo CSV

La lectura de datos emplea el elemento `csv.DictReader`, el cual mapea automáticamente la fila de encabezados como las claves de un diccionario para cada registro. Durante la iteración de carga, es necesario realizar una conversión explícita al tipo de dato entero mediante la función `int()` sobre los campos numéricos de población y superficie, ya que por defecto el formato CSV lee toda la información como cadena de texto (*strings*). Sin esta conversión, sería imposible realizar ordenamientos exactos o cálculos estadísticos.

Ambas funciones (*cargar* y *guardar*) manejan de forma segura la liberación de recursos en el sistema operativo mediante el uso de administradores de contexto (Context Managers a través de la sentencia *with open*), asegurando el correcto cierre del archivo incluso ante anomalías imprevistas.

Módulo modificar_registros.py

Concentra los algoritmos destinados a alterar el dataset, gestionando la incorporación de nuevas entidades geográficas y la edición de las existentes.

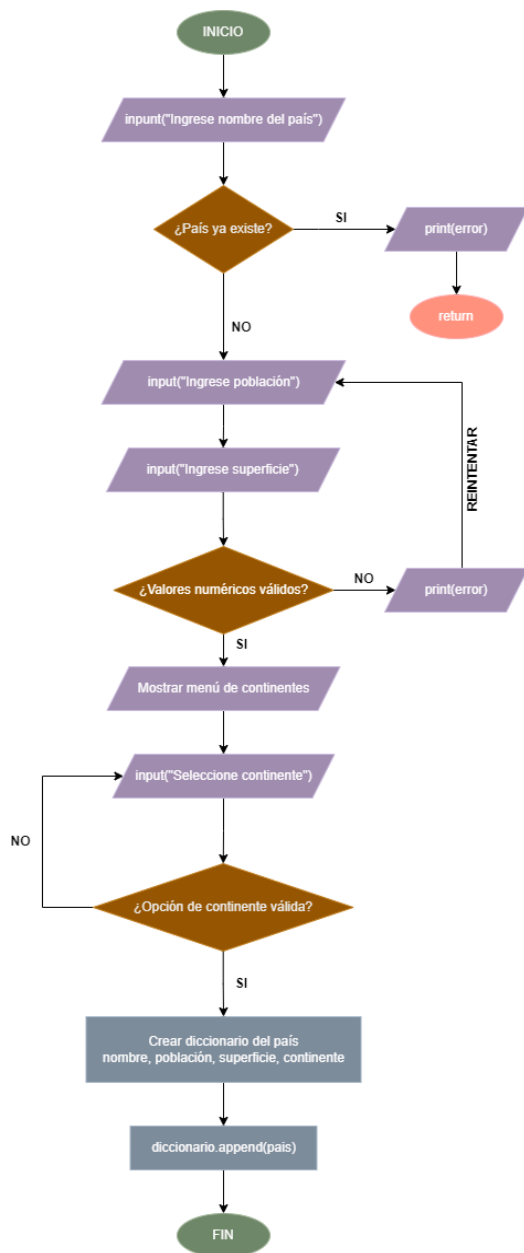


Fig 8: Diagrama de flujo de la función *agregar_pais*

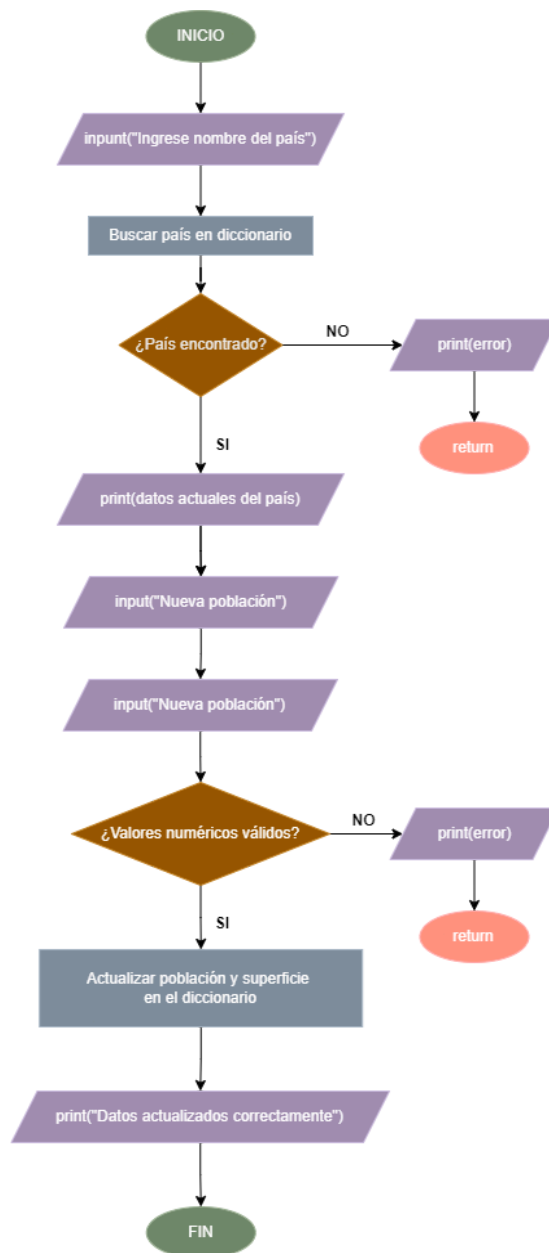


Fig 9: Diagrama de flujo de la función *actualizar_pob_sub*

```
Su eleccion: 1

Ingrese nombre de pais a agregar: Francia
Ingrese la poblacion actual de Francia (ingrese solamente numeros): 6000000
Ingrese la superficie de Francia (ingrese solamente numeros): 122112
Ingrese el continente donde se encuentra Francia:
1. África
2. América
3. Asia
4. Europa
5. Oceanía
```

Fig 10: Visualización de la función `agregar_pais()`

```
Ingrese nombre de pais a agregar: Argentina
Error. El pais ya se encuentra en los registros.

Bienvenido al sistema de gestion de datos geograficos. Ingrese la opcion para realizar la opcion que
desees:
```

Fig 11: Agregar país (duplicado)

```
===== ACTUALIZAR POBLACION Y SUPERFICIE DE UN PAIS =====
Ingrese el nombre del pais a actualizar: alemania
Datos actuales de Alemania: | Poblacion: 83149300 | Superficie: 357022 | Continente: Europa |
Ingrese la nueva poblacion: 2000000
Ingrese la nueva superficie (km²):
```

Fig 12: Visualización de la función `actualizar_pob_sup()`

Para la función `agregar_pais()`, la integridad de la información se resguarda mediante una verificación existencial previa. Utilizando la fuente nativa `any()`, se examina el dataset en tiempo lineal; si se detecta una coincidencia de nombres (el país ya existe), se interrumpe inmediatamente el flujo para prevenir registros duplicados.

Por su parte, la función `actualizar_pob_sup()` realiza una búsqueda indexada por contenido. Al hallar las coincidencias, se le despliega al usuario sus valores actuales y se procede a una sobre escritura directa sobre las claves `población` y `superficie` del diccionario referenciado, validando nuevamente mediante bloques `try-except` que los datos ingresados correspondan a magnitudes numéricas reales (precisamente números negativos).

Módulo consultas.py

En este módulo se agrupan las funciones que realizan algún tipo de consulta sobre los registros, sin modificar de ningún modo los mismos. Contiene la función `buscar_país()`, que permite ingresar un país, y realiza la búsqueda ya sea por coincidencia exacta, o por coincidencia parcial. La función `filtrar_país()` que permite hacer búsquedas avanzadas aplicando un filtro, ya sea por continente, por rango de población, o por rango de superficie. La función `ordenar_país()`, que permite ordenar todos los registros guardados y mostrarlos con criterio ya sea por nombre, por cantidad de población o por superficie. En todos los casos el ordenamiento puede realizarse de manera ascendente o descendente. Por último la función `mostrar_estadísticas()` permite mostrar las estadísticas permite mostrar un resumen global de los registros guardados de la siguiente manera: muestra el país con menor y mayor población respectivamente, y el promedio de población global. De igual forma con superficie, mostrando el país con mayor superficie, el país con menor superficie, y el promedio global. Por último, permite ver cuantos países hay cargados en cada continente.

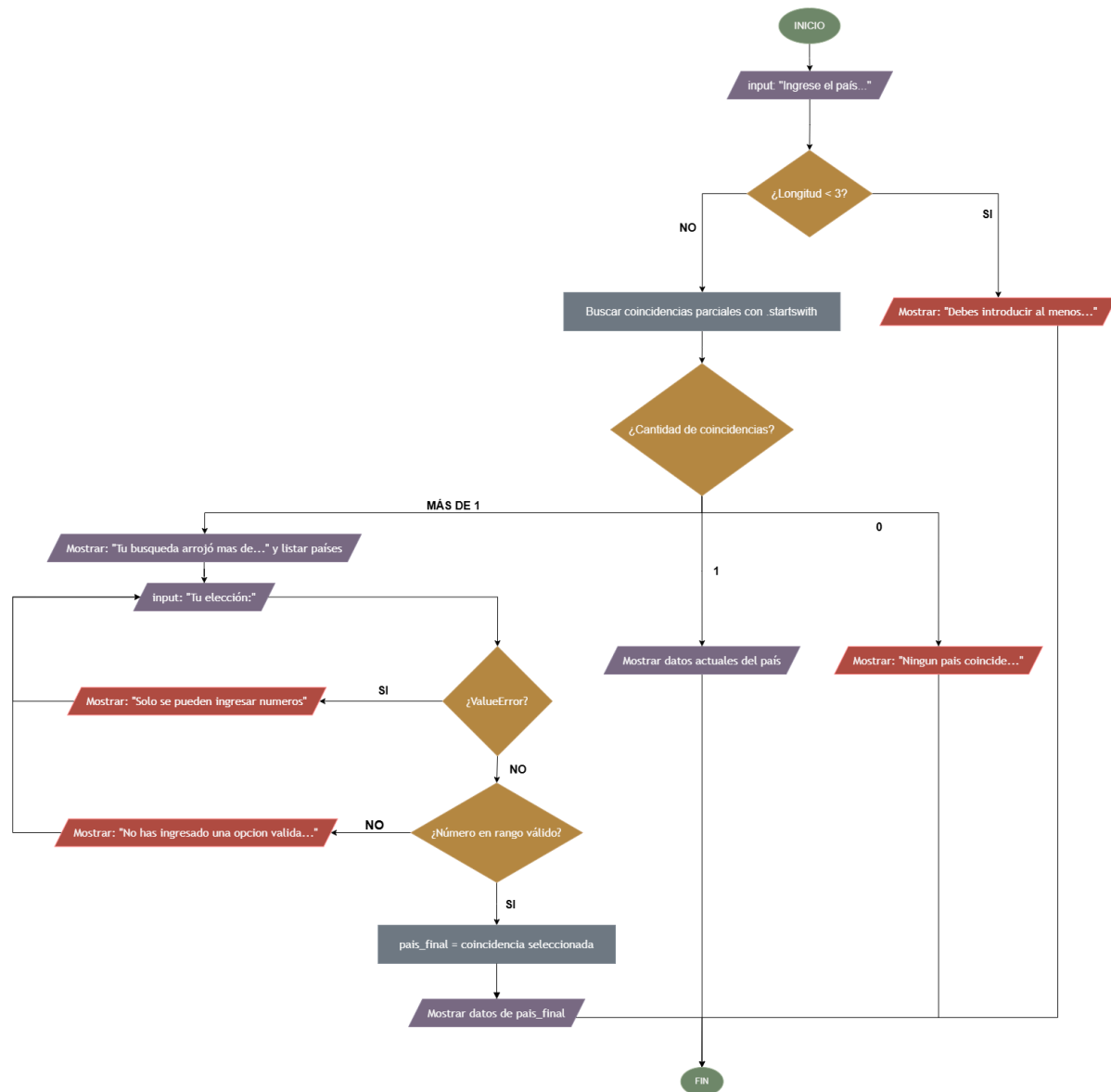


Fig 13: Diagrama de flujo de la función buscar_pais()

```
===== BUSCAR PAIS POR NOMBRE =====
```

```
Ingrese el país para buscar sus datos o introduce al menos las tres primeras letras para buscar coincidencias: arg
```

```
Tu búsqueda arrojó mas de un resultado. Selecciona el país de la lista para ver sus datos.
```

```
1 - Argentina
```

```
2 - Argelia
```

```
Tu elección: █
```

Fig 14: Búsqueda de país por nombre. La consola muestra menú de opciones de países que comienzan con las letras ingresadas por el usuario

buscar_pais(): Implementa una validación estricta de longitud mínima ($len() < 3$) para optimizar el rendimiento de la consulta. La comparación se flexibiliza convirtiendo la entrada a formato título (*.title()*) y evaluando coincidencias mediante el método *.startswith()*. Si el espectro de resultados es mayor a uno, se genera un menú dinámico numerado a partir de la lista de coincidencias parciales.

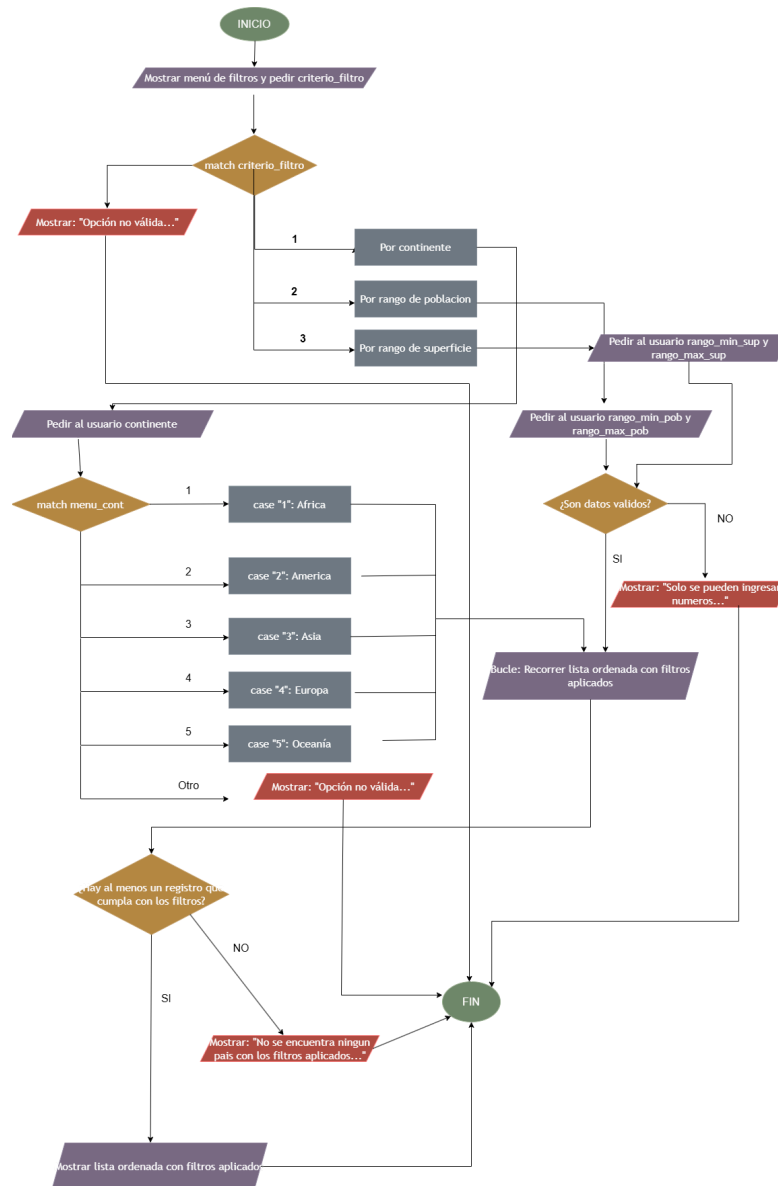


Fig 15: Diagrama de flujo de la función *filtrar_pais()*

```

===== FILTRAR PAÍSES - según el criterio elegido =====
1.Por Continente
2.Por Rango de Población
3.Por Rango de Superficie
=====

Seleccione el criterio: 

```

Fig 16: Menú de opciones para seleccionar criterio de filtro. Dentro de la función `filtrar_pais()`

`filtrar_pais()`: Utiliza una estructura condicional basada en evaluaciones booleanas sobre los atributos de los diccionarios, discriminando la búsqueda según tres ramas paramétricas independientes: pertenencia a cadena (Continentes) o acotamiento entre umbrales numéricos mínimos y máximos (Población y Superficie).

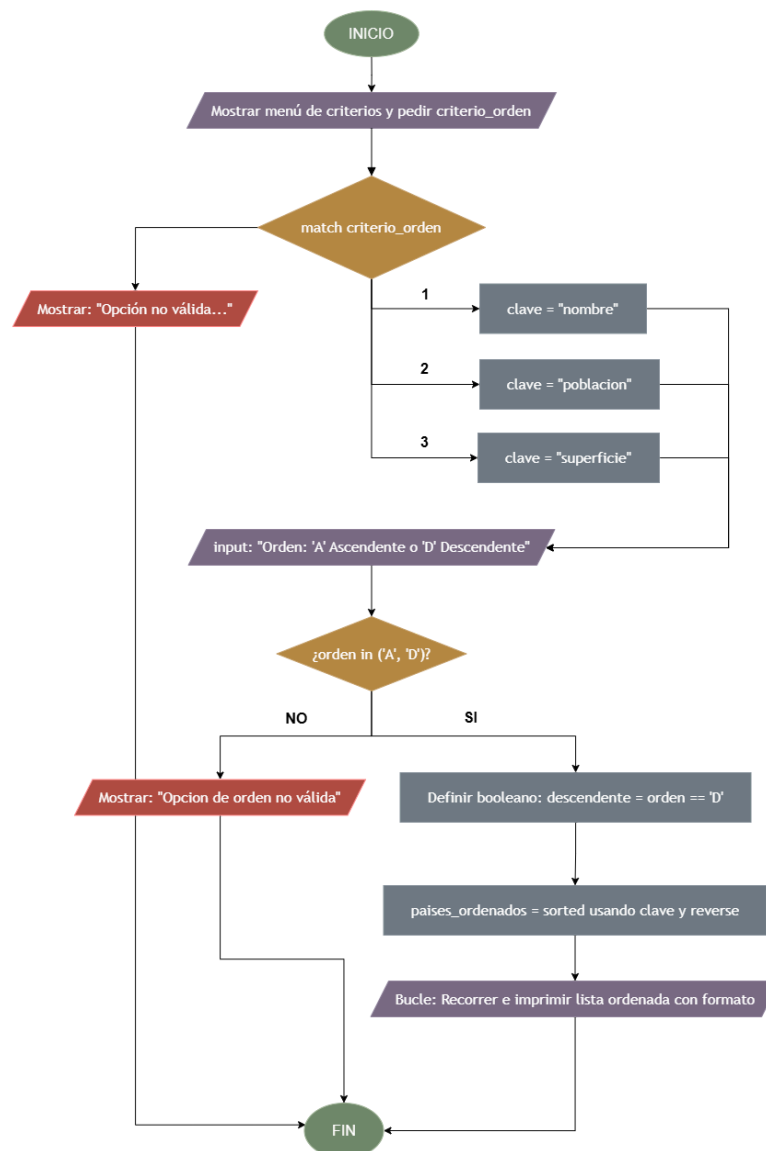


Fig 17: función `ordenar_pais()`

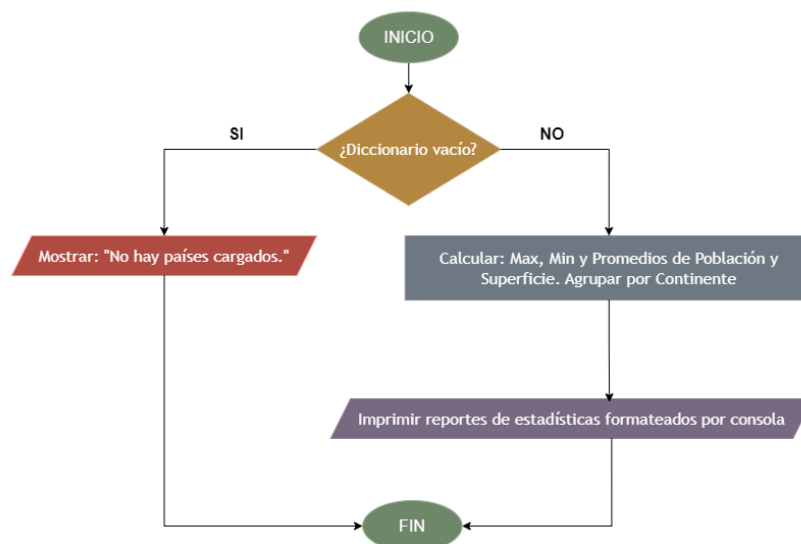
```

===== ORDENAR PAÍSES - según el criterio elegido =====
1.Por Nombre
2.Por Población
3.Por Superficie
=====
Seleccione el criterio: 1

```

Fig 18: Menú de opciones para seleccionar criterio de la función `ordenar_pais()`

`ordenar_pais()`: Delega la clasificación del dataset a la función nativa altamente optimizada `sorted()`. La abstracción se logra parametrizando dinámicamente la clave de ordenamiento mediante una función anónima (función Lambda) (`key=lambda p: p[clave]`), mientras que la dirección del orden (ascendente o descendente) se controla inyectando un valor booleano en el parámetro `reverse`.

Fig 19: Función `mostrar_estadisticas()`

```

===== ESTADÍSTICAS =====

Población:
Mayor: China(1,412,910,000)
Menor: Argelia(4,545,621)
Promedio: 191,220,185

Superficie:
Mayor: Rusia (17,125,191 km²)
Menor: Argelia (124,554 km²)
Promedio: 4,357,417 km²

Países por continente:
África: 2
América: 5
Asia: 4
Europa: 2
Oceanía: 1

```

Fig 20: Visualización de la función `mostrar_estadisticas()`

mostrar_estadisticas(): Ejecuta operaciones de reducción analítica sobre la colección de datos. Para hallar los valores extremos, se emplean de forma eficiente las funciones integradas *max()* y *min()* aplicando expresiones lambda como selectores de clave. El conteo de frecuencia de países por continente se resuelve elegantemente implementando un diccionario contador, el cual acumula dinámicamente las apariciones aprovechando el método seguro *.get(c, 0)* para inicializar claves inexistentes en tiempo de ejecución.

CONCLUSIONES

El desarrollo de este trabajo práctico integrador representó una experiencia de aprendizaje que excedió ampliamente lo estrictamente técnico. En primer lugar, es importante mencionar que durante el desarrollo del trabajo fueron surgiendo diversas dificultades que hubo que abordar. Al ser una de las primeras experiencias de trabajo con manejo de archivos que experimentaron los estudiantes, la implementación de un programa complejo, con varias funcionalidades distintas, fue todo un desafío a la hora de su desarrollo. Para superar estos obstáculos técnicos, la clave fue la adopción de una metodología de trabajo colaborativa. La división del sistema en módulos independientes (como *archivos.py*, *consultas.py*, entre otros) no solo facilitó la organización lógica del código, sino que permitió una distribución equitativa de las tareas, asignando roles claros que optimizaron la resolución de problemas específicos. La comunicación constante y la sincronización entre ambos integrantes fueron fundamentales para integrar las distintas funcionalidades de manera coherente.

Sin embargo, el contexto en el que se desarrolló este trabajo merece una reflexión particular. Estudiar de manera virtual y asincrónica implica una serie de desafíos que van más allá del contenido académico en sí. La distancia física, la comunicación mediada por pantallas y la necesidad de autogestionar los tiempos generan un escenario que demanda del estudiante una cuota considerable de autonomía y tolerancia a la incertidumbre. A esto se suma que, en ocasiones, la desorganización institucional introduce un factor de caos adicional: cambios de último momento en las consignas, información contradictoria o plazos poco claros que obligan a los estudiantes a reorganizarse sobre la marcha. Lejos de ser situaciones intencionales, estos inconvenientes son parte de una realidad compleja que enfrentan las instituciones. No obstante, paradójicamente, ese mismo caos se convierte en un espacio de encuentro: ante la confusión compartida, los estudiantes buscan a otros estudiantes, consultan, debaten, se organizan entre sí. En ese movimiento espontáneo emerge algo valioso: la construcción colectiva del conocimiento.

Esta dimensión social del aprendizaje, que en la presencialidad ocurre de manera casi natural, en la virtualidad requiere ser buscada activamente. Y cuando se encuentra, resulta igual de poderosa. Aprender con otros y de otros no es un complemento del proceso educativo, es parte constitutiva de él. Las dudas se resuelven en conjunto, las decisiones técnicas se discuten entre compañeros, los errores que se identifican en la revisión mutua, todo eso forma parte de una experiencia de aprendizaje que ningún material individual puede reemplazar.

En definitiva, este proyecto no solo permitió afianzar los conceptos técnicos aprendidos en Programación 1, sino que brindó una experiencia concreta sobre la gestión de un desarrollo de software real, donde la organización, la comunicación y el trabajo en equipo resultan tan críticos como la lógica de programación misma..

BIBLIOGRAFÍA

- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Listas en Python: Estructura de Datos Fundamental* [Presentación de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Listas: métodos y estructuras de datos* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Arreglos, Arrays y Arrays Bidimensionales* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Diccionarios* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Funciones* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Estructuras condicionales* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Algoritmos de ordenamiento* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Manipulación de archivos en Python* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Tecnicatura Universitaria en Programación a Distancia. (s.f.). *Archivos CSV, listas y diccionarios* [Material de cátedra, Programación I]. Universidad Tecnológica Nacional.
- ❖ Londero, O. (s.f.). *Toma de decisiones con lógica: Estructuras condicionales en algoritmos y Python* [Material didáctico, Programación I]. Universidad Tecnológica Nacional.
- ❖ Ivanisky, M. R. (s.f.). *Estadística y probabilidad II* [Material de cátedra, 4º año Prof. de Matemática]. Instituto Superior de Profesorado N° 1 "Manuel Leiva", Casilda.
- ❖ Python Software Foundation. (s.f.). *Built-in functions*. Python 3 Documentation.
<https://docs.python.org/3/library/functions.html>

ENLACES AL PROYECTO

- Repositorio del proyecto: <https://github.com/marcos-mag/TPI-Programacion-1-TUPaD>
- Video explicativo:  TPI-P1-VIDEO.mp4